



GameMatch AI

Video Game Recommendation

System

Submitted By:
Aman Negi

Roll Number: 33

Organization: JTP Project Round 2026

Technology Stack: FastAPI • React • PostgreSQL • Docker

Submission Date: (16th July 2026)



INDEX

1. Project Overview
2. System Architecture
3. Technology Stack
4. Project Structure
5. ETL Pipeline
6. Database Design
7. API Design
8. Recommendation Engine
9. Match Score Calculation
10. Frontend Architecture
11. Docker Architecture
12. Future Scope
13. Data Source
14. Screenshots
15. Conclusion



1. Project Overview

1.1 Introduction

GameMatch AI is a full-stack web application that provides personalized video game recommendations based on a user's preferences. Instead of displaying a generic list of popular games, the application analyzes multiple factors such as genres, platforms, gameplay tags, digital stores, minimum rating, release year, and maximum playtime to recommend games that best match the user's interests.

The project follows a complete software engineering workflow, beginning with data collection from the RAWG Video Games Database API through an ETL (Extract, Transform, Load) pipeline. The collected data is cleaned, transformed, and stored in a normalized PostgreSQL database. During runtime, the FastAPI backend retrieves data from the database, applies a custom weighted recommendation algorithm, and returns personalized recommendations to the React frontend.

In addition to implementing a recommendation system, the project demonstrates the integration of modern software engineering technologies and practices, including RESTful API development, relational database design, frontend development using React, Docker-based containerization, and version control with Git.

1.2 Project Objectives

The primary objectives of **GameMatch AI** are:

- Develop a personalized video game recommendation system based on user preferences using a custom weighted scoring algorithm.
- Build an ETL pipeline to extract, transform, and load game data from the RAWG API into a normalized PostgreSQL database.
- Develop RESTful APIs using FastAPI to process user requests and communicate with the frontend.
- Design a responsive and user-friendly interface using React to provide an interactive recommendation experience.
- Deploy the complete application using Docker and Docker Compose to ensure consistent and portable execution across different environments.

1.3 Project Features

The application provides the following features:

- Personalized game recommendations using a weighted scoring algorithm.
- Filtering based on genres, platforms, gameplay tags, stores, rating, release year, and playtime.
- Transparent recommendation scores with matched preference information.
- A normalized PostgreSQL database containing **15680** video games.
- Fast recommendation generation without requiring external API calls during runtime.
- Responsive and intuitive user interface built with React.
- Containerized deployment using Docker for easy setup and portability.



1.4 Why I Chose This Project

Video games have become one of the most popular forms of entertainment across all age groups. With thousands of games available on different platforms and in various genres, choosing the right game can often be confusing and time-consuming. Many players spend a significant amount of time browsing reviews, watching gameplay videos, or searching through online stores before deciding what to play.

This inspired me to build GameMatch AI, a video game recommendation system that suggests games based on a user's personal preferences, such as genres, platforms, tags, ratings, release year, playtime, and preferred stores. Instead of displaying the same popular games to everyone, the application provides recommendations that are tailored to each user's selections.

1.5 What Makes This Project Special?

1. Personalized Recommendations

Users receive recommendations based on their selected preferences instead of a fixed list of popular games.

2. Simple and Clean Interface

The application provides an easy-to-use interface where users can quickly select filters and get recommendations without unnecessary steps.

3. Fast Recommendations

Recommendations are generated within seconds using the local PostgreSQL database, providing a smooth user experience.

4. Flexible Search Options

Users can combine multiple filters such as genres, platforms, tags, rating, release year, playtime, and stores to find games that match their interests.

5. Clear Recommendation Results

Each recommended game displays important information such as rating, release year, genres, platforms, stores, and match score, helping users understand why it was recommended.

6. No Dependency on External APIs During Runtime

Since all game data is stored locally after the ETL process, users experience faster response times without being affected by external API delays or rate limits.

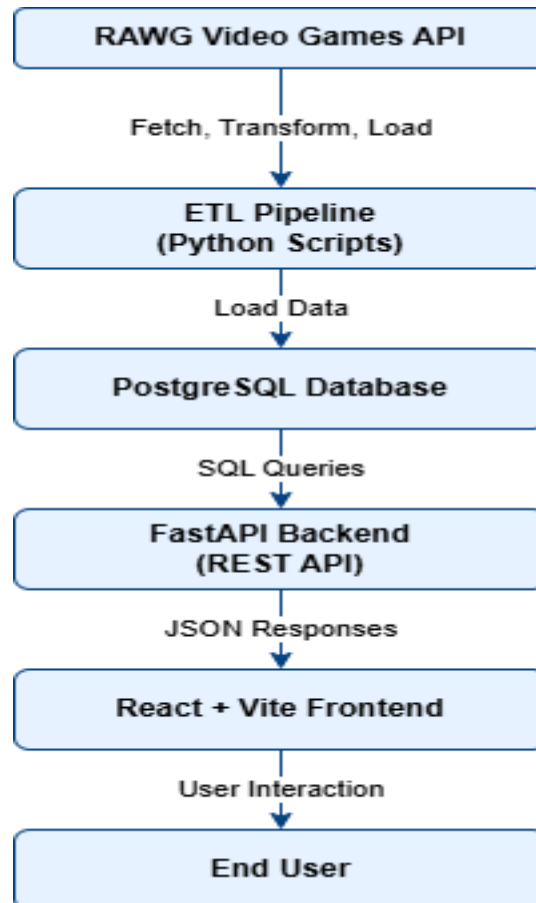


2. System Architecture

GameMatch AI follows a layered architecture where each component has a specific responsibility. The application consists of an ETL pipeline, a PostgreSQL database, a FastAPI backend, and a React frontend.

The ETL pipeline collects game data from the RAWG Video Games Database API, processes the data, and stores it in a PostgreSQL database. During application runtime, the FastAPI backend retrieves the required data from the database, processes the user's preferences using the recommendation engine, and returns the results to the React frontend. The frontend then displays the recommended games to the user.

The overall system architecture is illustrated below.



System Workflow

1. Game information is collected from the RAWG Video Games Database API through the ETL pipeline.
2. The extracted data is transformed and loaded into the PostgreSQL database.
3. The user selects preferences through the React frontend.
4. The frontend sends the selected preferences to the FastAPI backend.
5. The backend retrieves the required data from PostgreSQL and executes the recommendation algorithm.
6. The backend returns the ranked recommendations as JSON.
7. The React frontend displays the recommended games along with their details and match scores.



3. Technology Stack

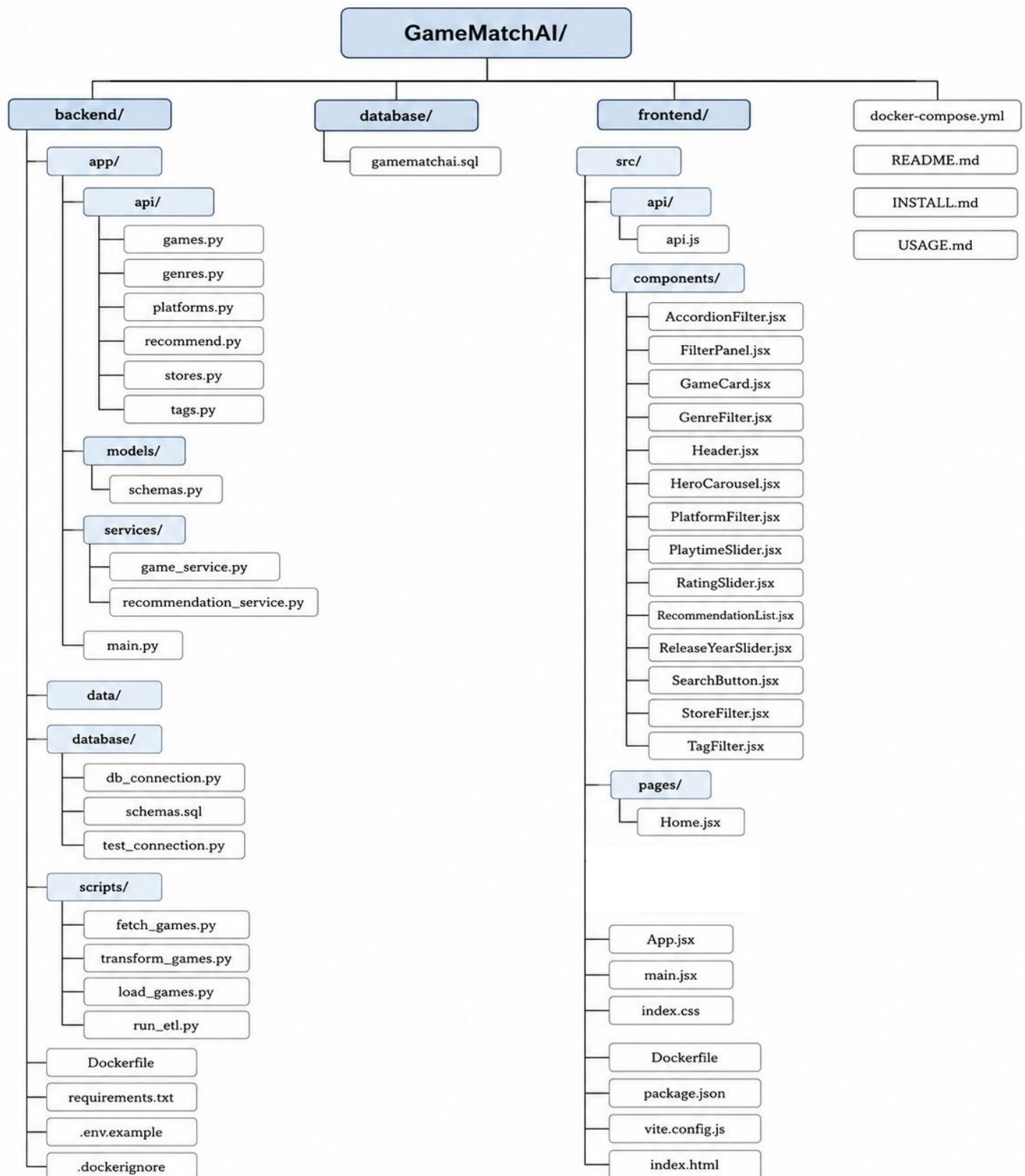
GameMatch AI was developed using the following technologies. Each technology was selected based on the requirements of the project.

Technology	Purpose
Python	Used for backend development and implementing the ETL pipeline.
FastAPI	Used to build REST APIs that handle communication between the frontend and the database.
React	Used to develop the user interface and display personalized game recommendations.
Vite	Used as the frontend build tool for React development.
Tailwind CSS	Used for styling the user interface and creating a responsive layout.
PostgreSQL	Used to store game data, genres, platforms, tags, stores, and screenshots in a relational database.
RAWG API	Used as the source of game data during the ETL process.
Docker	Used to containerize the frontend, backend, and database services.
Docker Compose	Used to manage and run all Docker containers with a single command.
Axios	Used by the React frontend to send HTTP requests to the FastAPI backend.
Git	Used for version control during project development.
GitHub	Used to host the project repository and documentation.
Visual Studio Code	Used as the primary development environment for writing and managing the project code.



4. Project Structure

The project is organized into separate backend, frontend, and database modules to maintain a clean and modular architecture. Each directory is responsible for a specific part of the application.



Directory / File	Description
backend/app/api/	Contains the FastAPI API endpoints responsible for handling HTTP requests and returning responses to the frontend.
backend/app/models/	Defines the Pydantic schemas used to validate incoming requests and structure API responses.
backend/app/services/	Contains the application's business logic. <code>game_service.py</code> retrieves game data, while <code>recommendation_service.py</code> implements the weighted recommendation algorithm.
backend/database/	Contains the PostgreSQL connection configuration, database schema, and connection testing files.
backend/scripts/	Contains the ETL pipeline scripts used to extract, transform, and load game data into PostgreSQL.
frontend/src/components/	Contains reusable React components such as filters, sliders, recommendation cards, and other user interface elements.
frontend/src/pages/	Contains the main application page (<code>Home.jsx</code>) that combines all frontend components.
frontend/src/api/	Handles communication between the React frontend and the FastAPI backend using Axios.
database/	Stores the PostgreSQL database dump (<code>gamematchai.sql</code>) used to initialize the database automatically.
docker-compose.yml	Defines and manages the PostgreSQL, FastAPI, and React containers using Docker Compose.
README.md	Contains the complete project overview, architecture, implementation details, and technical documentation.
INSTALL.md	Provides step-by-step instructions for installing and setting up the project.
USAGE.md	Explains how to start the application and use its features.

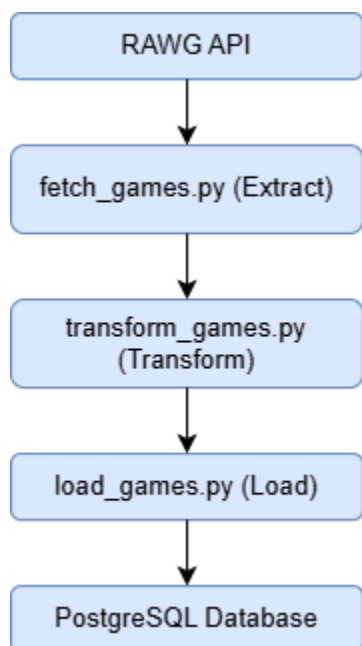


5. ETL Pipeline

The GameMatch AI application uses an **ETL (Extract, Transform, Load)** pipeline to collect, prepare, and store game data before it is used by the recommendation engine. Instead of requesting data from the RAWG API every time a user searches for recommendations, the application retrieves the data once, processes it, and stores it in a local PostgreSQL database.

This approach improves application performance, reduces dependency on external APIs during runtime, and provides faster recommendation results.

ETL Workflow



File	Purpose
<code>fetch_games.py</code>	Retrieves game information from the RAWG Video Games Database API and stores the raw response as JSON files.
<code>transform_games.py</code>	Extracts the required fields, handles missing values, removes duplicate metadata, organizes relationships, and prepares the data for PostgreSQL.
<code>load_games.py</code>	Loads the transformed data into PostgreSQL while maintaining the relationships between tables.
<code>run_etl.py</code>	Executes the complete ETL pipeline by running the Extract, Transform, and Load stages in sequence.



Data Preparation

During the transformation stage, the raw API response is processed and organized before being loaded into the PostgreSQL database. The following operations are performed:

- Extracts only the fields required by the application.
- Removes duplicate genres, platforms, tags, and stores.
- Converts nested JSON data into a normalized relational structure.
- Creates many-to-many relationship mappings between games and their associated genres, platforms, tags, and stores.
- Organizes the processed data into separate JSON files for efficient loading into the database.

Benefits of the ETL Pipeline

- Stores all game data locally in PostgreSQL.
- Eliminates dependency on the RAWG API during normal application usage.
- Improves recommendation speed by querying the local database.
- Makes it easy to refresh the database by running the ETL pipeline again when new game data is required.



6. Database Design

GameMatch AI uses **PostgreSQL** as its relational database to store and manage video game information. The database is designed using normalization principles to reduce data redundancy and maintain relationships between different entities.

Instead of storing all information in a single table, related data such as genres, platforms, tags, stores, and screenshots are stored in separate tables and connected through many-to-many relationship tables. This structure improves data consistency and simplifies database maintenance.

Database Overview

The PostgreSQL database contains **15680 video games** collected through the ETL pipeline from the **RAWG Video Games Database API**.

The database stores:

- Game information
- Genres
- Platforms
- Gameplay tags
- Digital stores
- Screenshots
- Relationship data between games and their metadata

Database Tables

Table	Purpose
games	Stores basic information about each game, including its name, release date, rating, Metacritic score, playtime, ESRB rating, and cover image.
genres	Stores all unique game genres.
platforms	Stores all supported gaming platforms.
tags	Stores gameplay tags associated with games.
stores	Stores digital game stores
screenshots	Stores screenshots associated with each game.
game_genres	Maps games to one or more genres.



Table	Purpose
game_platforms	Maps games to their supported platforms.
game_tags	Maps games to gameplay tags.
game_stores	Maps games to the stores where they are available.

Database Design Highlights

- Uses a normalized relational database structure.
- Reduces duplicate data by separating metadata into individual tables.
- Uses many-to-many relationship tables to connect games with genres, platforms, tags, and stores.
- Maintains referential integrity using primary and foreign keys.
- Supports efficient SQL queries required by the recommendation engine.



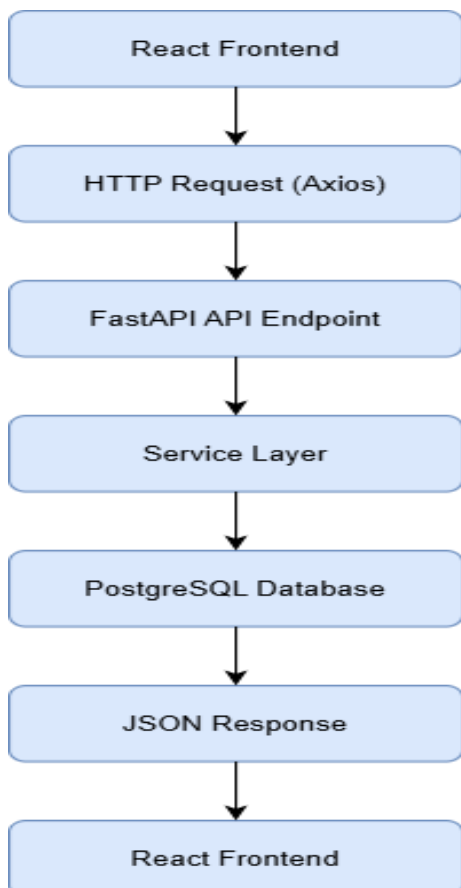
7. API Design

The backend of GameMatch AI is developed using **FastAPI** and exposes RESTful APIs that enable communication between the React frontend and the PostgreSQL database. These APIs allow the frontend to retrieve game-related data and generate personalized recommendations based on the user's selected preferences.

Available API Endpoints

Method	Endpoint	Description
GET	/games	Retrieves the list of available games from the database.
GET	/genres	Returns all available game genres.
GET	/platforms	Returns all supported gaming platforms.
GET	/tags	Returns all available gameplay tags.
GET	/stores	Returns all available digital game stores.
POST	/recommend	Generates personalized game recommendations based on the user's selected preferences.

API Workflow



Request Validation

FastAPI uses **Pydantic schemas** to validate incoming requests before they reach the recommendation engine. This ensures that the required data is received in the correct format and reduces the possibility of invalid inputs.

The recommendation request includes:

- Genres
- Platforms
- Tags
- Minimum Rating
- Release Year
- Maximum Playtime
- Preferred Stores

API Response

The recommendation API returns a list of games in **JSON** format. Each recommendation includes:

- Game ID
- Game Name
- Cover Image
- Release Year
- Rating
- Genres
- Platforms
- Tags
- Stores
- Recommendation Match Score
- Matched Preferences

Benefits of the API Design

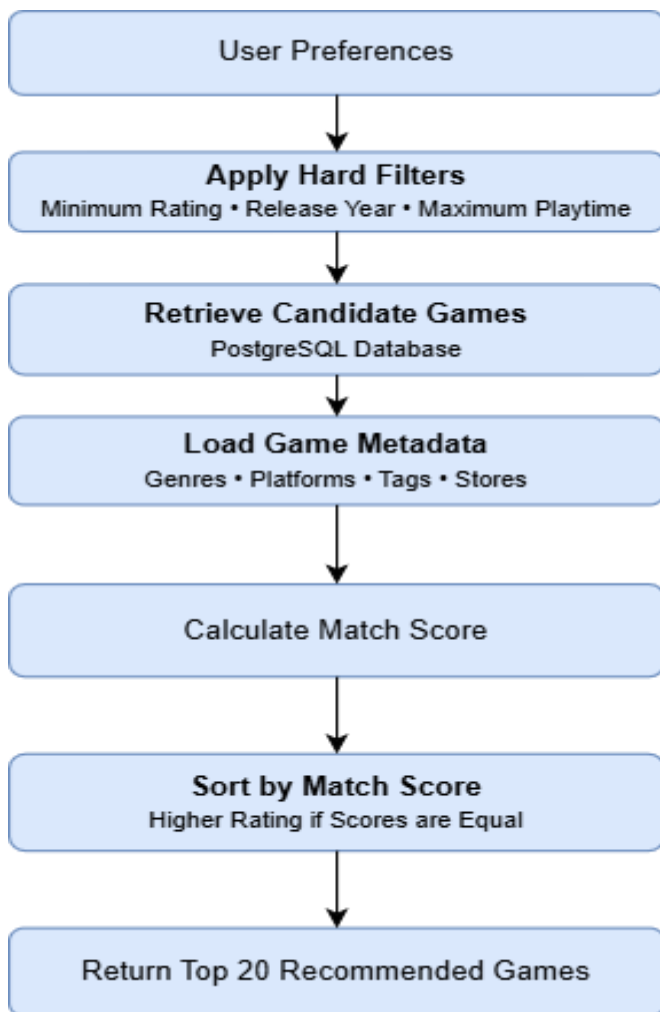
- Provides a clear separation between the frontend and backend.
- Uses RESTful APIs for communication.
- Returns data in JSON format for easy processing by the React frontend.
- Uses Pydantic schemas to ensure consistent request validation and response formatting.



8. Recommendation Engine

The recommendation engine is the core component of GameMatch AI. It generates personalized game recommendations by comparing the user's selected preferences with the game data stored in the PostgreSQL database.

Instead of using a machine learning model, the application uses a **custom weighted scoring algorithm**. Each user preference contributes a predefined number of points to the final recommendation score, making the recommendation process transparent and easy to understand.



Recommendation Process

The recommendation process consists of the following steps:

1. Candidate Filtering

- Games are filtered using the selected minimum rating, release year, and maximum playtime.
- This reduces the number of games that need to be evaluated.

2. Metadata Retrieval

- The genres, platforms, tags, and stores for the remaining games are retrieved from the



database.

3. Match Score Calculation

- Each candidate game is compared with the user's selected preferences.
- A weighted score out of **100** is calculated.

4. Ranking

- Games are sorted according to their recommendation score.
- If two games have the same score, the game with the higher user rating is ranked first.

5. Recommendation Results

- The application returns the top 20 highest-ranked games along with their recommendation scores and matched preferences.

Scoring Criteria

Criteria	Weight
Genre	35
Platform	25
Tags	20
Rating	5
Release Year	5
Playtime	3
Store	7

Maximum Recommendation Score = 100

Why a Weighted Recommendation Algorithm?

A weighted scoring approach was selected because it is simple, transparent, and easy to explain. Each recommendation score is calculated using predefined weights, allowing users to understand why a game was recommended. The algorithm can also be easily adjusted in the future by modifying the scoring weights without changing the overall system design.



9. Match Score Calculation

Each candidate game is assigned a Recommendation Match Score out of 100 points. The score is calculated by comparing the game's attributes with the user's selected preferences. Each criterion contributes a predefined number of points based on its assigned weight.

Scoring Criteria

Criteria	Maximum Score	Description
Genre	35	Awarded proportionally based on the number of matched genres.
Platform	25	Awarded proportionally based on the number of matched platforms.
Tags	20	Awarded proportionally based on the number of matched tags.
Rating	5	Awarded if the game's rating meets or exceeds the user's minimum rating.
Release Year	5	Awarded if the game was released in or after the selected year.
Playtime	3	Awarded if the game's playtime is within the selected maximum playtime.
Store	7	Awarded if the game is available on one of the selected stores.

Maximum Recommendation Score = 100

Example

Suppose the user selects the following preferences:

- **Genres:** Action, RPG, Adventure, Shooter
- **Platforms:** PC, PlayStation 5
- **Tags:** Open World, Multiplayer, Fantasy, Story Rich
- **Minimum Rating:** 4
- **Release Year:** 2020
- **Maximum Playtime:** 40 Hours
- **Preferred Store:** Steam

A candidate game has the following attributes:

- **Genres:** Action, RPG, Puzzle
- **Platforms:** PC, Xbox
- **Tags:** Open World, Fantasy, Survival, Multiplayer



- **Rating:** 4.5
- **Release Year:** 2022
- **Playtime:** 30 Hours
- **Available Stores:** Steam, Epic Games

Match Score Breakdown

Criteria	Explanation	Calculation	Score
Genre	The user selected 4 genres, and the game matches 2 of them (Action and RPG).	$(2 \div 4) \times 35$	17.5
Platform	The user selected 2 platforms, and the game matches 1 (PC).	$(1 \div 2) \times 25$	12.5
Tags	The user selected 4 tags, and the game matches 2 (Open World and Fantasy).	$(2 \div 4) \times 20$	10
Rating	The game's rating (4.5) meets the user's minimum rating (4).	Full Score	5
Release Year	The game was released in 2022, which satisfies the selected year (2020).	Full Score	5
Playtime	The game's playtime (30 hours) is within the selected maximum (40 hours).	Full Score	3
Store	The game is available on the selected store (Steam).	Full Score	7

Final Recommendation Score

$$17.5 + 12.5 + 10 + 5 + 5 + 3 + 7 = 60$$

The calculated score is rounded to the nearest whole number before being displayed as the **Recommendation Match Score**.

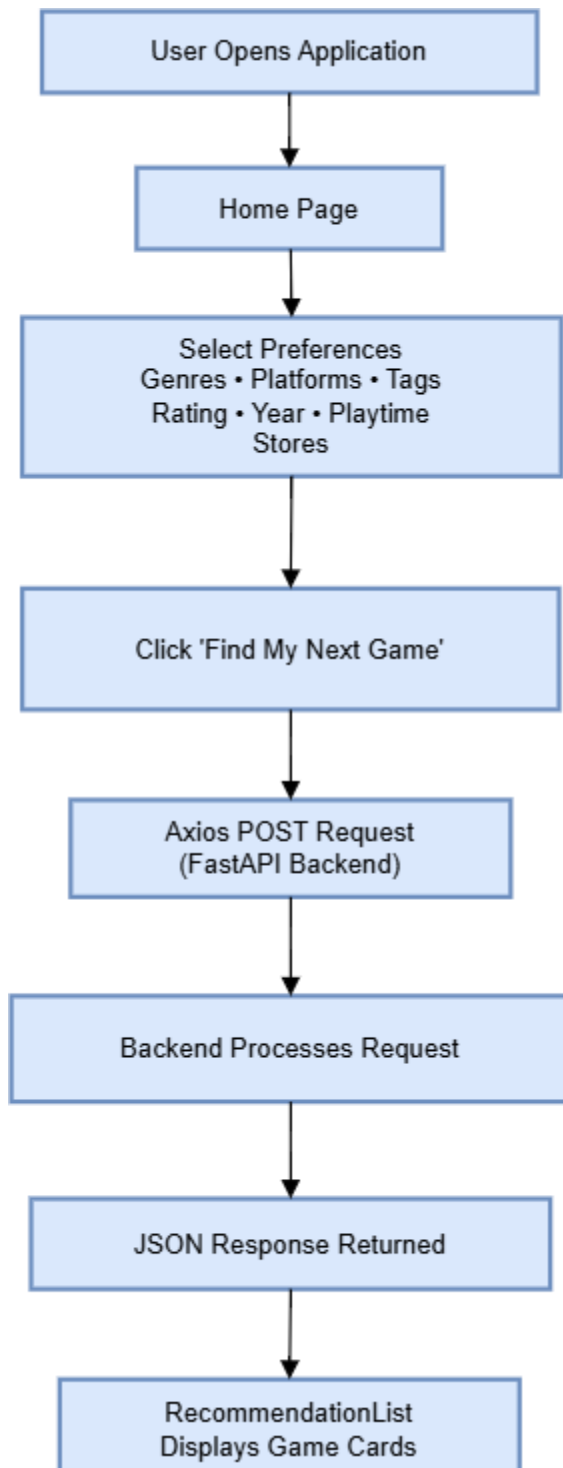


10. Frontend Architecture

The frontend of GameMatch AI is developed using **React** and follows a component-based architecture. Each component is responsible for a specific part of the user interface, making the application easier to maintain and extend.

The frontend allows users to select their preferences, sends the selected data to the FastAPI backend, and displays personalized game recommendations returned by the recommendation engine.

Frontend Workflow



File	Purpose
main.jsx	Entry point of the React application.
App.jsx	Loads the main application component.
Home.jsx	Displays the main application page.
Header.jsx	Displays the application header and hero section.
HeroCarousel.jsx	Shows featured game banners.
FilterPanel.jsx	Organizes all user preference filters.
AccordionFilter.jsx	Creates expandable filter sections.
GenreFilter.jsx	Selects preferred game genres.
PlatformFilter.jsx	Selects preferred gaming platforms.
TagFilter.jsx	Selects gameplay tags.
StoreFilter.jsx	Filters games by digital stores.
RatingSlider.jsx	Selects the minimum game rating.
ReleaseYearSlider.jsx	Filters games by release year.
PlaytimeSlider.jsx	Sets the maximum preferred playtime.
SearchButton.jsx	Sends the recommendation request to the backend.
RecommendationList.jsx	Displays recommended games.
GameCard.jsx	Displays detailed information for each game.
api.js	Handles API communication using Axios.
index.css	Contains global application styles and animations.
App.css	Default Vite stylesheet (not used for custom UI).
index.html	Main HTML entry point of the application.

Frontend Features

- Interactive preference filters.
- Responsive user interface.
- Recommendation cards displaying detailed game information.
- Component-based design for better code organization and reusability.

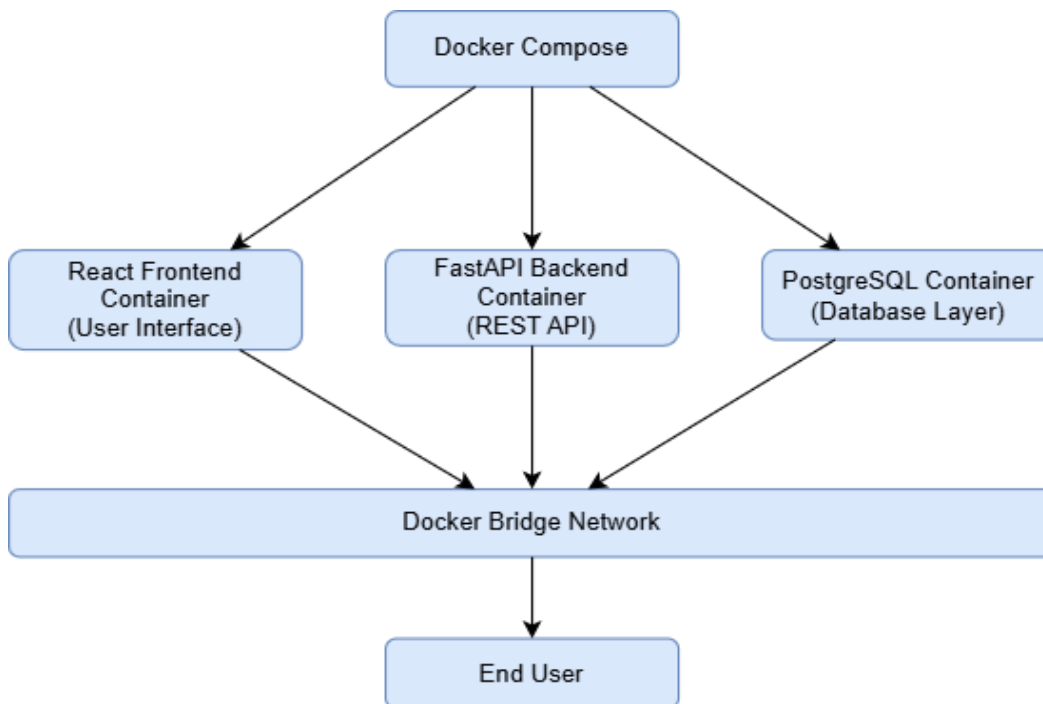


11. Docker Architecture

GameMatch AI is containerized using **Docker** and **Docker Compose**. The application consists of three separate containers: one for the React frontend, one for the FastAPI backend, and one for the PostgreSQL database.

Using Docker ensures that the application runs in a consistent environment across different systems and simplifies the setup process by allowing all services to be started with a single command.

Docker Architecture



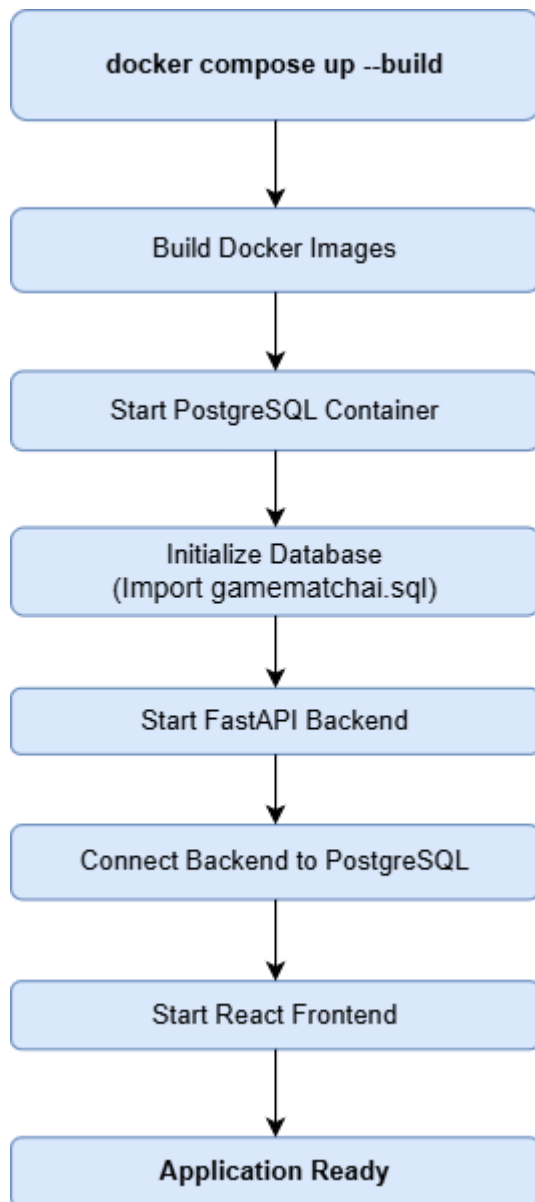
Docker Containers

Container	Purpose
gamematchai_postgres	Stores the game database and is automatically initialized using the provided SQL dump.
gamematchai_backend	Processes recommendation requests, communicates with PostgreSQL, and returns API responses.
gamematchai_frontend	Provides the user interface for selecting preferences and displaying recommendations.

Container Communication The containers communicate through the Docker bridge network created by Docker Compose



Docker Workflow



Advantages of Using Docker

- Runs the application in a consistent environment.
- Keeps the frontend, backend, and database isolated in separate containers.
- Simplifies project setup with a single `docker compose up --build` command.
- Reduces dependency and version conflicts across different operating systems.
- Makes the project easier to deploy and share.



12. Future Scope

Although GameMatch AI meets the current project requirements, there are several opportunities for future improvement:

- **Machine Learning Integration** – Enhance the recommendation engine using machine learning for more personalized recommendations.
- **Similar Game Search** – Allow users to search for a game and receive recommendations for similar games.
- **Larger Game Database** – Import more games from the RAWG API to increase the variety of recommendations.
- **Store Links and Prices** – Display direct links to digital stores along with game prices and discounts.
- **Additional Game Information** – Include trailers, screenshots, descriptions, developers, publishers, and system requirements.
- **Improved UI/UX** – Enhance the interface with better navigation, animations, themes, and interactive features.
- **More Search Filters** – Add filters such as ESRB rating, multiplayer support, developer, publisher, and supported languages.
- **Performance Optimization** – Improve database queries and recommendation performance as the dataset grows.



13. Data Source

GameMatch AI uses the **RAWG Video Games Database API** as the source of game data.

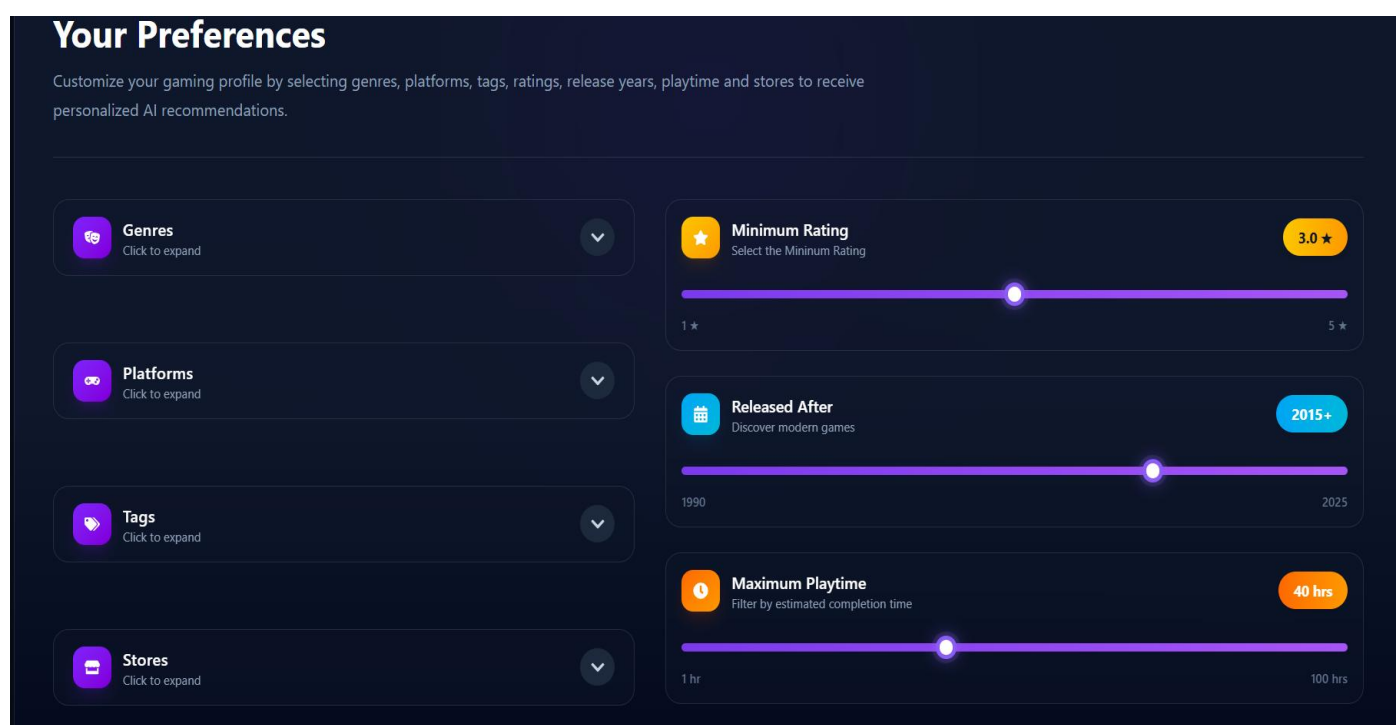
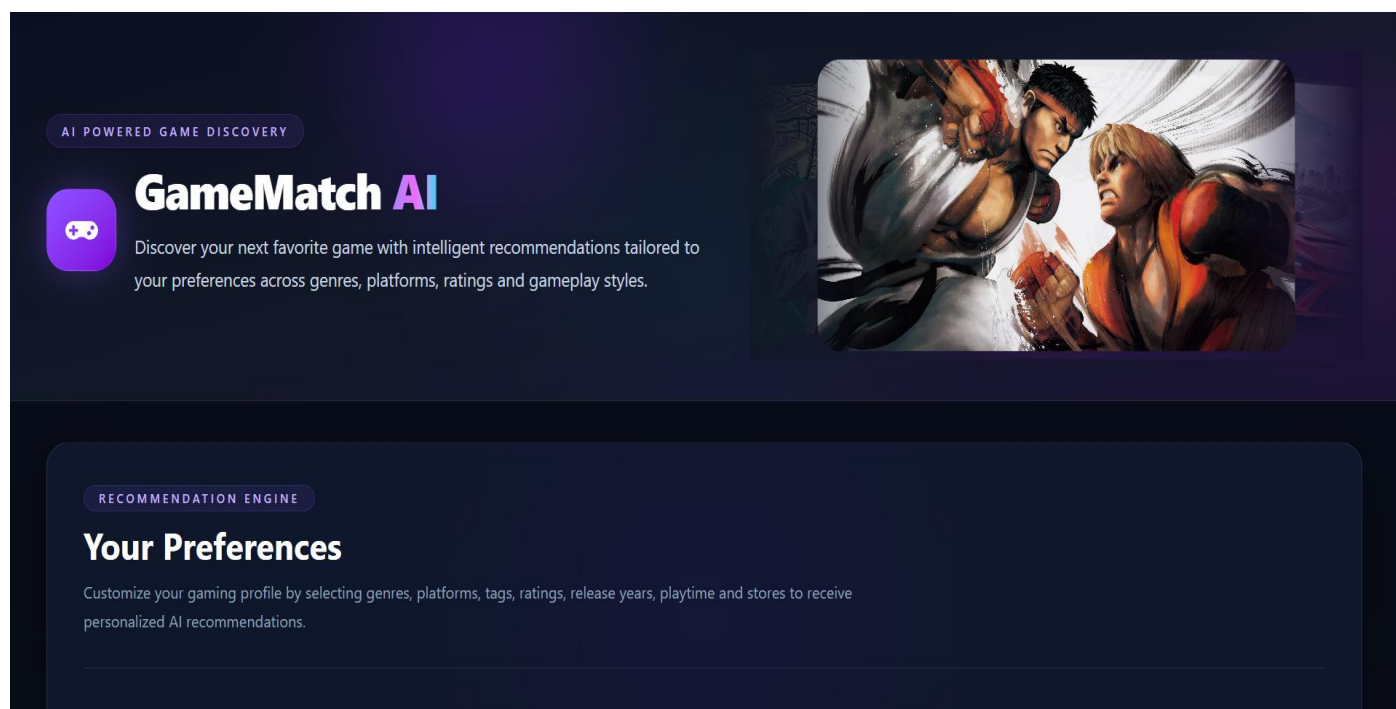
The ETL pipeline retrieves publicly available information such as game titles, genres, platforms, tags, stores, ratings, release dates, playtime, and screenshots. The extracted data is then transformed and stored in a local PostgreSQL database.

After the ETL process is completed, the application generates all recommendations using the local database without making external API calls during runtime.

Source: <https://rawg.io/apidocs>



14. Screenshots



Genres

Click to collapse

^

Action

✓

Adventure

+

Arcade

+

Board Games

+

Card

+

Platforms

Click to collapse

^

PlayStation 2

+

PlayStation 3

+

PlayStation 4

✓

PlayStation 5

✓

PS Vita

+

Tags

Click to collapse

^

single

SELECTED TAGS

controller-support ×

great-graphics ×

Great Soundtrack ×

Sandbox ×

Singleplayer ×

Single player only

+

Singleplayer

✓

Stores

Click to collapse

^

Nintendo Store

+

PlayStation Store

✓

Steam

✓

Xbox 360 Store

+

Xbox Store

+

★

Minimum Rating

Select the Minimum Rating

3.0 ★

1 ★

5 ★

📅

Released After

Discover modern games

2015+

1990

2025

🕒

Maximum Playtime

Filter by estimated completion time

40 hrs

1 hr

100 hrs





92% MATCH

Cyberpunk 2077

★ 4.2 2020

GENRES Action Shooter RPG

PLATFORMS Nintendo Switch PlayStation 4 PC Xbox Series S/X PlayStation 5 Xbox One

TAGS #Singleplayer #Atmospheric #Great Soundtrack #RPG #Story Rich

STORES PlayStation Store Xbox Store Steam Epic Games GOG

Matched On

Action Shooter PlayStation 4 PC PlayStation 5 Singleplayer Great Soundtrack Sandbox
PlayStation Store Steam Epic Games



88% MATCH

Mass Effect: Legendary Edition

★ 4.6 2021

GENRES Action Shooter Adventure RPG

PLATFORMS PlayStation 5 Xbox Series S/X PlayStation 4 PC Xbox One

TAGS #Singleplayer #Steam Achievements #Full controller support #Atmospheric #Great Soundtrack

STORES Xbox Store Steam PlayStation Store

Matched On

Action Shooter PlayStation 5 PlayStation 4 PC Singleplayer Great Soundtrack Steam
PlayStation Store



88% MATCH

DOOM Eternal

★ 4.4 2020

GENRES Action Shooter

PLATFORMS PC PlayStation 4 Xbox One Nintendo Switch PlayStation 5

TAGS #Singleplayer #Multiplayer #Atmospheric #Great Soundtrack #Co-op

STORES GOG PlayStation Store Xbox Store Nintendo Store Steam

Matched On

Action Shooter PC PlayStation 4 PlayStation 5 Singleplayer Great Soundtrack
PlayStation Store Steam



88% MATCH

Hitman 3

★ 4.3 2021

GENRES Action Shooter Adventure

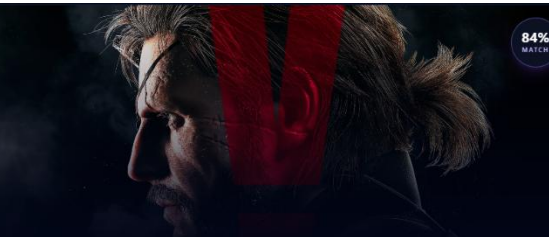
PLATFORMS PC Xbox One PlayStation 4 Xbox Series S/X PlayStation 5 Nintendo Switch

TAGS #Singleplayer #Steam Achievements #Full controller support #Steam Cloud #Third Person

STORES Xbox Store Nintendo Store PlayStation Store Epic Games Steam

Matched On

Action Shooter PC PlayStation 4 PlayStation 5 Singleplayer Sandbox PlayStation Store
Epic Games Steam



84% MATCH

Metal Gear Solid V: The Phantom Pain

★ 4.2 2015

GENRES Action Shooter

PLATFORMS PlayStation 4 PC Xbox One PlayStation 3 Xbox 360

TAGS #Singleplayer #Steam Achievements #Multiplayer #Full controller support #Steam Cloud

STORES PlayStation Store Steam Xbox 360 Store Xbox Store

Matched On

Action Shooter PlayStation 4 PC Singleplayer Great Soundtrack Sandbox PlayStation Store
Steam



84% MATCH

Control

★ 4.1 2019

GENRES Action Shooter Adventure

PLATFORMS Xbox Series S/X PlayStation 5 Xbox One PlayStation 4 PC

TAGS #Singleplayer #Full controller support #Atmospheric #Third Person #Sci-fi

STORES PlayStation Store GOG Steam Xbox Store Epic Games

Matched On

Action Shooter PlayStation 5 PlayStation 4 PC Singleplayer PlayStation Store Steam
Epic Games



15. Conclusion

GameMatch AI is a full-stack video game recommendation system that combines data engineering, backend development, database management, frontend development, and containerization into a single application.

The project uses an ETL pipeline to collect game data from the RAWG Video Games Database API, transforms and stores the data in a normalized PostgreSQL database, and generates personalized recommendations using a custom weighted scoring algorithm. The React frontend provides a simple and interactive interface through which users can select their preferences and view recommendation results.

The application is fully containerized using Docker, allowing the frontend, backend, and database to run together in a consistent environment with minimal setup.

This project provided practical experience in designing REST APIs with FastAPI, working with relational databases, implementing an ETL pipeline, building reusable React components and integrating multiple technologies.

The modular architecture also allows the project to be extended in the future by adding new recommendation techniques, additional filters, more game data, and enhanced user interface features.



日本サードパーティ株式会社

